

```

// Project UID 1d9f47bfc76643019cfbf037641defe1

#include "Player.h"
#include <algorithm>
#include <cassert>

class Simple :public Player {
private:
    std::string nameP;
    std::vector<Card> hand{};

public:
    Simple(std::string namePlayer) : nameP(namePlayer) {}
    //EFFECTS returns player's name
    virtual const std::string& get_name() const {
        return nameP;
    }

    //REQUIRES player has less than MAX_HAND_SIZE cards
    //EFFECTS adds Card c to Player's hand
    virtual void add_card(const Card& c) {
        hand.push_back(c);
        std::sort(hand.begin(), hand.end());
    }

    //REQUIRES round is 1 or 2
    //MODIFIES order_up_suit
    //EFFECTS If Player wishes to order up a trump suit then return true and
    // change order_up_suit to desired suit. If Player wishes to pass, then do
    // not modify order_up_suit and return false.
    virtual bool make_trump(const Card& upcard, bool is_dealer,
        int round, std::string& order_up_suit) const {
        int trumps = 0;
        int size = static_cast<int>(hand.size());
        if (round == 1) {
            for (int i = 0; i < size; ++i) {
                if (hand[i].is_face() && hand[i].is_trump(upcard.get_suit())) {
                    ++trumps;
                }
            }
            if (trumps >= 2) {
                order_up_suit = upcard.get_suit();
                return true;
            }
        }

        if (round == 2) {
            std::string nextSuit = Suit_next(upcard.get_suit());
            for (int i = 0; i < size; ++i) {
                if (hand[i].is_face() && hand[i].is_trump(nextSuit)) {
                    ++trumps;
                }
            }
            if (trumps >= 1) {
                order_up_suit = nextSuit;
                return true;
            }
            if (is_dealer) {
                order_up_suit = nextSuit;
            }
        }
    }
}

```

```

        return false;
    }

    //REQUIRES Player has at least one card
    //EFFECTS Player adds one card to hand and removes one card from hand.
    virtual void add_and_discard(const Card& upcard) {

        std::string trump = upcard.get_suit();// make_trump??
        int lowest = 0;
        int size = static_cast <int>(hand.size());
        hand.push_back(upcard);
        std::sort(hand.begin(), hand.end());
        for (int i = 0; i < size; ++i) {
            if (Card_less(hand[i], hand[lowest], trump)) {
                lowest = i;
            }
        }
        hand.erase((hand.begin() + lowest));
        // remove min card with cardless function
    }

    //REQUIRES Player has at least one card, trump is a valid suit
    //EFFECTS Leads one Card from Player's hand according to their strategy
    // "Lead" means to play the first Card in a trick. The card
    // is removed the player's hand.
    virtual Card lead_card(const std::string& trump) {
        std::sort(hand.begin(), hand.end());
        int size = static_cast <int>(hand.size());
        int highest = 3;
        bool allTrump = true;
        for (int i = 0; i < size; ++i) {
            if (hand[i].get_suit() != trump) { // for--> of all trump --> if 1
not trump
                allTrump = false;
            }
        }
        for (int i = 0; i < size; ++i) {
            if (allTrump == true && Card_less(hand[highest], hand[i], trump)) {
                highest = i;
            }
            if (allTrump == false) {
                if (hand[i].get_suit() != trump &&
                    Card_less(hand[highest], hand[i], trump)) {
                    highest = i;
                }
            }
        }
        Card H = hand[highest];
        hand.erase((hand.begin() + highest));
        return H;
    }

    //REQUIRES Player has at least one card, trump is a valid suit
    //EFFECTS Plays one Card from Player's hand according to their strategy.
    // The card is removed from the player's hand.
    virtual Card play_card(const Card& led_card, const std::string& trump) {
        int highest = 4;
        int size = static_cast <int>(hand.size());
        int lowest = 0;
        std::sort(hand.begin(), hand.end());
        for (int i = 0; i < size; ++i) {

            if (hand[i].get_suit() == led_card.get_suit()) {

```

```

        if (Card_less(hand[highest], hand[i], led_card, trump)) {
            highest = i;
        }
    }
    Card H = hand[i];
    hand.erase((hand.begin() + i));
    return H;
}
for (int i = 0; i < size; ++i) {
    if (hand[i].get_suit() != led_card.get_suit()) {
        if (Card_less(hand[i], hand[lowest], led_card, trump)) {
            lowest = i;
        }
    }
}
Card L = hand[i];
hand.erase((hand.begin() + i));
return L;
}
}

};

class Human :public Player {
private:
    std::string nameP;
    std::vector<Card> hand;

public:
    Human(std::string namePlayer) : nameP(namePlayer) {}
    //EFFECTS returns player's name
    virtual const std::string& get_name() const {
        return nameP;
    }

    //REQUIRES player has less than MAX_HAND_SIZE cards
    //EFFECTS adds Card c to Player's hand
    virtual void add_card(const Card& c) {
        if (hand.size() < MAX_HAND_SIZE) {
            hand.push_back(c);
        }
        std::sort(hand.begin(), hand.end());
    }

    //REQUIRES round is 1 or 2
    //MODIFIES order_up_suit
    //EFFECTS If Player wishes to order up a trump suit then return true and
    // change order_up_suit to desired suit. If Player wishes to pass, then do
    // not modify order_up_suit and return false.
    virtual bool make_trump(const Card& upcard, bool is_dealer,
        int round, std::string& order_up_suit) const {
        int size = static_cast <int>(hand.size());

        if (round == 1 || round == 2) {
            for (int i = 0; i < size; ++i) {
                std::cout << "Human player " << nameP << "'s " << "hand: "
                    << "[" << i << "]" << " " << hand[i] << std::endl;
            }
            std::cout << "Human player " << nameP <<
                ", please enter a suit, or 'pass':";
        }
    }
};

```

```

        std::string answer;
        std::cin >> answer;
        if (answer == "pass") {
            std::cout << nameP << " passes" << std::endl;
        }
        if (answer == upcard.get_suit()) {
            upcard.get_suit() = order_up_suit;
            std::cout << nameP << " orders up" << upcard.get_suit() <<
std::endl;
        }
    }
    return false;
}

//REQUIRES Player has at least one card
//EFFECTS Player adds one card to hand and removes one card from hand.
virtual void add_and_discard(const Card& upcard) {
    bool is_dealer = true;
    int size = static_cast<int>(hand.size());
    std::sort(hand.begin(), hand.end());
    int round{};
    std::string order_up_suit;
    if (is_dealer && round == 1 && upcard.get_suit() == order_up_suit) {
        for (int i = 0; i < size; ++i) {
            std::cout << "Human player " << nameP << "'s " << "hand: "
                << "[" << i << "]" << hand[i] << std::endl;
        }
        std::cout << "Discard upcard : [-1]" << std::endl;
        std::cout << "Human player " << nameP <<
            ", please select a card to discard :";
        int answer;
        std::cin >> answer;
        std::cout << std::endl;
        if (answer != -1) {
            hand.push_back(upcard);
        }

        for (int i = 0; i < size; ++i) {
            if (answer == i) {
                hand.erase((hand.begin() + i));
            }
        }
    }
}

//REQUIRES Player has at least one card, trump is a valid suit
//EFFECTS Leads one Card from Player's hand according to their strategy
// "Lead" means to play the first Card in a trick. The card
// is removed the player's hand.
virtual Card lead_card(const std::string& trump) {
    std::sort(hand.begin(), hand.end());
    int size = static_cast<int>(hand.size());
    for (int i = 0; i < size; ++i) {
        std::cout << "Human player " << nameP << "'s " << "hand: " <<
            "[" << i << "]" << hand[i] << std::endl;
    }
    std::cout << "Human player " << nameP << ", please select a card to
discard :";
    int answer;
    std::cin >> answer;
    std::cout << std::endl;
    std::cout << hand[answer] << " led by " << nameP << std::endl;
}

```

```

        hand.erase((hand.begin() + answer));
        return hand[answer];
    }

    //REQUIRES Player has at least one card, trump is a valid suit
    //EFFECTS Plays one Card from Player's hand according to their strategy.
    // The card is removed from the player's hand.
    virtual Card play_card(const Card& led_card, const std::string& trump) {
        int size = static_cast<int>(hand.size());
        std::sort(hand.begin(), hand.end());
        for (int i = 0; i < size; ++i) {
            std::cout << "Human player " << nameP << "'s " << "hand: " <<
                "[" << i << "]" << hand[i] << std::endl;
        }
        std::cout << "Human player " << nameP << ", please select a card to
discard :";
        int answer;
        std::cin >> answer;
        hand.erase((hand.begin() + answer));
        return hand[answer];
    }

};

//EFFECTS: Returns a pointer to a player with the given name and strategy
//To create an object that won't go out of scope when the function returns,
//use "return new Simple(name)" or "return new Human(name)"
//Don't forget to call "delete" on each Player* after the game is over
Player* Player_factory(const std::string& name, const std::string& strategy) {

    if (strategy == "Simple") {
        return new Simple(name);
    }
    if (strategy == "Human") {
        return new Human(name);
    }

    return nullptr;
}

//EFFECTS: Prints player's name to os
std::ostream& operator<<(std::ostream& os, const Player& p) {
    os << p.get_name();
    return os;
}

```